Module : GIT

Day 1: Git & GitHub Basics

What is Git?

Git is a distributed version control system that allows multiple developers to work on a project simultaneously without overwriting each other's changes. It tracks modifications to files over time, enabling you to revert to specific versions when needed.

Why Use Git? (Importance)

- **Version Control**: Tracks changes to your codebase over time
- **Collaboration**: Multiple developers can work on the same project simultaneously
- **Backup**: Serves as a backup system for your code
- **Experimentation**: Create branches to test new features without affecting the main code
- **History**: Maintains a complete history of who changed what and when

Centralized Code Base Importance

A centralized code repository (like GitHub) provides:

- Single source of truth for project code
- Seamless collaboration between team members
- Access control and security features
- Backup in case of local system failure
- Simplified onboarding for new team members

Installing Git

Windows:

1. Download the installer from git-scm.com
2. Run the installer and follow the prompts
3. Open Command Prompt or PowerShell and type `git --version` to verify installation

Mac:

1. Install using Homebrew: `brew install git`
2. Or download from git-scm.com
3. Open Terminal and type `git --version` to verify installation

Linux (Ubuntu/Debian):

```
sudo apt update
sudo apt install git
git --version
```

Setting Up GitHub Account

1. Go to github.com
2. Click on "Sign up" and fill in the required information
3. Verify your email address
4. Set up a profile picture and bio (optional but recommended)

Configuring Git on Your Local Computer

After installing Git, configure your identity:

```
git config --global user.name "Your Name"
git config --global user.email "youremail@example.com"
```

Basic Git Commands

`git init`

Initializes a new Git repository in your current directory.

```
mkdir my-project
cd my-project
git init
```

Importance: This creates the foundation for tracking your project with Git.

`git add`

Stages changes for commit.

```
# Add a specific file
git add filename.txt
```

```
# Add all files
git add .
```

Importance: Allows you to select which changes you want to include in your next commit.

`git commit`

Saves the staged changes to the repository with a descriptive message.

```
git commit -m "Add Text file"
```

Importance: Creates a snapshot of your code at a specific point in time that you can revert to if needed.

`git push`

Upload's your local commits to a remote repository (like GitHub).

```
git push origin main
```

Importance: Share your changes with others and backs up your work remotely.

`git pull`

Downloads and integrates changes from a remote repository to your local repository.

```
git pull origin main
```

Importance: Keeps your local repository updated with changes made by others.

`git status`

Shows the status of changes as untracked, modified, or staged.

```
git status
```

Importance: Helps you understand the current state of your repository.

`git clone`

Creates a local copy of a remote repository.

```
git clone https://github.com/username/repository.git
```

Importance: Allows you to start working with an existing project.

Setting Up a Project and Using These Commands

Create a new repository on GitHub:

- Go to GitHub and click "New repository"
- Name your repository (e.g., "my-first-project")
- Add a description (optional)
- Choose public or private
- Click "Create repository"

Initialize your local repository and connect it to GitHub:

```
mkdir my-first-project
cd my-first-project
git init
echo "# My First Project" > README.md
git add README.md
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/yourusername/my-first-project.git
git push -u origin main
```

Make changes and push them:

```
echo "This is a test project to learn Git." >> README.md
git add README.md
git commit -m "Update README with description"
git push origin main
```

Pull changes from remote repository:

```
git pull origin main
```

Day 2: Advanced Git

Building on Day 1 Commands

Let's reinforce what we learned with a simple workflow example:

```
# Add a new feature
touch feature.txt
echo "New feature implementation" > feature.txt
git add feature.txt
git commit -m "Add new feature"
git push origin main
```

Branching Strategies

Branching allows you to diverge from the main line of development without affecting it.

Common Branching Strategies:

1. **Gitflow**:
    - `main` branch for production code
    - `develop` branch for development
    - Feature branches for new features
    - Release branches for preparing releases
    - Hotfix branches for emergency fixes
2. **GitHub Flow**:
    - `main` branch always contains deployable code
    - Feature branches for new work
    - Pull requests for review before merging
3. **Trunk-Based Development**:
    - Short-lived feature branches

- Frequent merges to main branch
- Emphasis on continuous integration

Creating Branches

Create and switch to a new branch

```
git checkout -b feature-login
```

Alternative method (newer Git versions)

```
git switch -c feature-login
```

Importance: Allows you to work on features in isolation without affecting the main codebase.

Ways to Create Branches

From current state:

```
git branch new-branch
git checkout new-branch

# OR in one command
git checkout -b new-branch
```

1. **From a specific commit**:

```
git checkout -b new-branch commit-hash
```

2. **From a tag**:

```
git checkout -b new-branch tag-name
```

3. **From another branch**:

```
git checkout -b new-branch existing-branch
```

Managing Branches

Listing Branches

```
# List local branches
git branch

# List remote branches
git branch -r


# List all branches (local and remote)
git branch -a
```

Switching Between Branches

```
git checkout branch-name

# OR (newer Git versions)
git switch branch-name
```

Deleting Branches

```
# Delete a local branch
git branch -d branch-name

# Force delete a branch
git branch -D branch-name

# Delete a remote branch
git push origin --delete branch-name
```

Stashing Changes

Stashing allows you to save uncommitted changes temporarily.

```
# Stash changes
git stash

# List stashes
git stash list
```

```
# Apply the most recent stash
git stash apply

# Apply a specific stash
git stash apply stash@{2}

# Apply and remove the most recent stash
git stash pop

# Remove a stash
git stash drop stash@{0}

# Create a new branch from a stash
git stash branch new-branch stash@{0}
```

Importance: Lets you switch contexts quickly without committing incomplete work.

Revert and Reset

Reverting Commits

Creates a new commit that undoes changes from a previous commit.

```
git revert commit-hash
```

Importance: Safely undoes changes while preserving history.

Resetting Commits

Moves the HEAD pointer and optionally changes files.

```
# Soft reset (keeps changes staged)
git reset --soft HEAD~1

# Mixed reset (unstages changes)
git reset HEAD~1

# Hard reset (discards changes)
git reset --hard HEAD~1
```

Importance: Allows you to clean up local commits before pushing.

Rebasing

Rebasing replays your changes on top of another branch.

While on your feature branch

```
git rebase main
```

Importance: Creates a cleaner, linear project history.

Interactive Rebase

```
git rebase -i HEAD~3
```

Importance: Allows you to clean up and modify commits before sharing.

Merging

Combining changes from different branches.

```
# While on the main branch
git merge feature-branch
```

Importance: Incorporates completed work into the main codebase.

Merge Strategies

1. **Fast-forward merge**: When there are no new changes in the target branch
2. **Recursive merge**: Default when branches have diverged

Squash merge: Combines all commits from source branch into one commit

```
 git merge --squash feature-branch
```

Handling Merge Conflicts

Conflicts occur when Git can't automatically merge changes.

When a conflict occurs, Git marks the conflicting areas:

```
<<<<<<< HEAD
Current branch code
=======
Incoming branch code
>>>>>>> feature-branch
```

1. Edit the file to resolve the conflict:

```
# Open the file in your editor
nano conflicted-file.txt

# Resolve conflicts, then save and close

# Mark as resolved
git add conflicted-file.txt

# Continue the merge or rebase
git merge --continue
# OR
git rebase --continue
```

Public vs Private Repositories

Public Repositories

- **Visibility**: Code is accessible to everyone
- **Use cases**: Open-source projects, educational material, portfolios
- **Benefits**: Community contributions, visibility, feedback

Private Repositories

- **Visibility**: Code is only visible to you and collaborators
- **Use cases**: Proprietary code, client work, sensitive projects
- **Benefits**: Security, IP protection, controlled access

Working with Collaborators

Adding Collaborators to GitHub Repository

1. Go to your repository on GitHub

2. Click on "Settings"
3. Select "Manage access"
4. Click "Invite a collaborator"
5. Enter their GitHub username or email

Collaboration Workflow

1. Collaborator clones the repository
2. Creates a branch for their feature
3. Makes changes and commits
4. Pushes branch to GitHub
5. Creates a pull request
6. Owner reviews changes
7. Owner merges or requests changes

Pull Requests

```
# Create a pull request (done on GitHub)
# After pushing your branch
git push origin feature-branch
# Then go to GitHub to create the PR
```

Importance: Facilitates code review and discussion before changes are merged.

Real-World Collaboration Scenarios

Scenario 1: Feature Development

```
# Developer 1: Create feature branch
git checkout -b feature-user-login
# Make changes, commit, and push
git push origin feature-user-login
# Create pull request on GitHub

# Developer 2: Review pull request on GitHub
# Make comments, suggest changes

# Developer 1: Address feedback
git add modified-files
git commit -m "Address PR feedback"
```

```
git push origin feature-user-login

# After approval, merge on GitHub or locally
```

Scenario 2: Hotfix for Production

```
# Create hotfix branch from main
git checkout main
git checkout -b hotfix-security-issue

# Fix issue, commit and push
git add fixed-files
git commit -m "Fix security vulnerability"
git push origin hotfix-security-issue

# Create emergency PR, get quick review
# Merge to main and then to develop
```

Scenario 3: Merge Conflicts in Team Environment

```
# Two developers modify the same file

# Developer 1 pushes first
git push origin feature-a

# Developer 2 tries to push
git push origin feature-b
# Gets rejection due to conflicts

# Developer 2 resolves
git pull origin main
# Resolves conflicts
git add resolved-files
git commit -m "Resolve merge conflicts"
git push origin feature-b
```

Git Commands Cheat Sheet

Setup and Configuration

```
git config --global user.name "Your Name"           # Set username
git config --global user.email "email@example.com"  # Set email
git config --list                                   # List configuration
```

Repository Initialization

```
git init                                            # Initialize repository
git clone <url>                                     # Clone repository
```

Basic Snapshotting

```
git status                                          # Show status
git add <file>                                      # Stage file
git add .                                           # Stage all files
git commit -m "message"                             # Commit with message
git commit -a -m "message"                          # Stage and commit all changes
```

Branching and Merging

```
git branch                                          # List branches
git branch <branch-name>                            # Create branch
git checkout <branch-name>                          # Switch branch
git checkout -b <branch-name>                       # Create and switch
git merge <branch-name>                             # Merge branch into current
git branch -d <branch-name>                         # Delete branch
git switch <branch-name>                            # Switch branch (new syntax)
git switch -c <branch-name>                         # Create and switch (new syntax)
```

Remote Repositories

```
git remote -v                                       # List remotes
git remote add <name> <url>                         # Add remote
git fetch <remote>                                  # Fetch from remote
git pull <remote> <branch>                          # Fetch and merge
git push <remote> <branch>                          # Push to remote
git push -u <remote> <branch>                       # Push and set upstream
```

Inspection and Comparison

```
git log                                    # View commit history
git log --oneline                          # Compact history
git diff                                   # Show unstaged changes
git diff --staged                          # Show staged changes
git diff <commit1> <commit2>               # Compare commits
git show <commit>                          # Show commit details
```

Undoing Changes

```
git restore <file>                         # Discard changes (new)
git restore --staged <file>                # Unstage changes (new)
git reset <file>                           # Unstage file
git reset --soft HEAD~1                    # Undo last commit, keep changes staged
git reset HEAD~1                           # Undo last commit, unstage changes
git reset --hard HEAD~1                    # Undo last commit, discard changes
git revert <commit>                        # Create new commit that undoes changes
```

Stashing

```
git stash                                  # Stash changes
git stash list                             # List stashes
git stash apply                            # Apply stash
git stash pop                              # Apply and drop stash
git stash drop                             # Delete stash
```

Advanced Operations

```
git rebase <branch>                        # Rebase current branch
git rebase -i HEAD~<n>                     # Interactive rebase
git cherry-pick <commit>                   # Apply commit to current branch
git tag <name>                             # Create tag
git tag -a <name> -m "message"             # Create annotated tag
git bisect start                           # Start binary search for bug
git blame <file>                           # Show who changed what and when
```

Collaboration

```
git fetch --all                                 # Fetch from all remotes
git pull --rebase                               # Pull with rebase instead
of merge
git push --force-with-lease                     # Safe force push
git request-pull                                # Create pull request
```

Cleanup

```
git clean -n                                    # Show what would be deleted
git clean -f                                    # Delete untracked files
git gc                                          # Garbage collection
```

This cheat sheet covers most common Git commands you'll need in your development workflow.